과제 #1

M1522.006700 확장형 고성능 컴퓨팅 (001)

2021-15531 컴퓨터공학부 김인호
September 21, 2025

1 Compilation Process

1.1 Preprocessing

(a) `stdio.h`
파일 경로: `/usr/include/stdio.h`, 파일의 라인 수: 875

`math.h`
파일 경로: `/usr/include/math.h`, 파일의 라인 수: 1341

(b) `sqrt` 디렉토리에서 `gcc -E sqrt.c -o sqrt.i`를 실행한 후 각 함수를 찾은 결과는 아래와 같다.
scanf 부분:

```
extern int scanf (const char *__restrict __format, ...) ;
```

printf 부분:

```
extern int printf (const char *__restrict __format, ...);
```

sqrt 부분:

```
extern double sqrt (double __x) __attribute__ ((__nothrow__ , __leaf__));
```

(c) Preprocess된 결과에는 실제 구현이 들어있지 않다. 그 이유는 헤더 파일에는 함수에 대한 선언만 들어있기 때문이다. 실제 구현이 들어있는 라이브러리 파일은 linking 과정에서 object file과 결합된다.

1.2 Compilation

(a) `sqrt.c`가 위치한 `sqrt` 디렉토리에서 `gcc -c sqrt.c`를 실행한다.

(b) `sqrt.o`가 위치한 디렉토리에서 명령 `file sqrt.o`를 실행하면 아래와 같은 결과가 나온다.

```
sqrt/sqrt.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

이를 통해 `sqrt.o`의 파일 포맷이 ELF (Executable and Linkable Format)라는 것을 알 수 있다.

1.3 Linking

(a) 1.1 (c)에서 언급한 것처럼, `sqrt.c`를 컴파일한 `sqrt.o`에는 `sqrt` 등의 함수에 대한 실제 구현 코드가 없기 때문에 이러한 에러가 발생한다.

이 에러를 해결하려면, `-lm`을 명령의 뒷부분에 추가하여 최종 실행파일을 생성하는 과정에서 제곱근 함수가 구현된 별도의 라이브러리(`libm.a`)를 link하여야 한다. 따라서, 올바르게 컴파일하여 `sqrt`라는 이름의 최종 실행파일을 생성하는 명령은 다음과 같다: `gcc sqrt.o -o sqrt -lm`

(b) 아래와 같은 결과를 확인할 수 있다.

```
● shpc028@login2:~/shpc-homework/hw1/sqrt$ gcc sqrt.o -o sqrt -lm
● shpc028@login2:~/shpc-homework/hw1/sqrt$ ./sqrt 2
  1.41421356
● shpc028@login2:~/shpc-homework/hw1/sqrt$ ./sqrt 100000000
  10000.00000000
● shpc028@login2:~/shpc-homework/hw1/sqrt$ ./sqrt 20250922
  4500.10244328
```

2 C Programming

2.1 Shift

가독성을 위해 4비트 단위로 끊어 작성하였다.

(a) `1111 1111 1111 1111 1111 1111 1111 0000`

(b) `1111 1111 1111 1111 1111 1111 1111 1100`

(c) `0011 1111 1111 1111 1111 1111 1111 1100`

(d) Arithmetic Shift는 MSB를 부호로 간주하여, 오른쪽 시프트를 할 때 채워지는 부분을 원래 수의 부호(MSB)와 같도록 한다. 이는 부호를 유지하고, 음수의 시프트도 양수처럼 나눗셈(Euclidean division)과 같은 효과를 내도록 한다. 예를 들어, `a >> b`는 `a`를 $2^{\mathtt{b}}$로 나눈 몫이다.

Logical Shift는 채워지는 부분을 반드시 0으로 한다. 이는 연산 대상을 부호 있는 수가 아닌 단순히 비트패턴으로 간주하는 연산으로, 음수의 보수 표현을 대상으로 logical shift를 하면 나눗셈의 의미를 가지지 않게 된다.

2.2 Convert

변환 대상 타입과 `unsigned int`와 같이 부호 없는 타입을 멤버로 가진 union을 선언하여 형 변환(type punning)을 수행하면 된다. 실제 구현 코드는 함께 제출한 `convert.c`에 있다.

3 클러스터 사용 연습

(a) `$ sinfo`

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
class1       up       1:00      5    mix a[00-04]
class1       up       1:00      7   idle a[05-11]
```

Slurm 노드와 파티션 현황을 확인할 수 있다. 출력에는 `class1`이라는 파티션의 `a[00-11]` 노드를 볼 수 있으며, 파티션과 각 노드의 상태 등을 확인할 수 있다.
파티션의 사용 가능 상태(AVAIL)가 `up`이므로, 파티션에 새로운 작업이 추가될 수 있고 작업이 노드에 할당되어 실행될 수 있다.
작업의 최대 시간 제한(TIMELIMIT)은 1분으로 설정되어 있는 것을 알 수 있다.
각 노드의 상태(STATE)는 5개의 노드가 부분적으로 사용 중(`mix`)이고, 7개의 노드가 쉬고 있는 중(`idle`)임을 알 수 있다.

(b) `$ squeue`

```
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
```

Slurm scheduling queue에 배정된 작업 목록을 확인할 수 있다. 위 명령 결과에서는 내용 없이 필드 이름만 출력되었으므로, 현재 할당된 작업이 없음을 의미한다. 만약 `loop`이라는 프로그램을 `srun`으로 계산 노드에 올린 후 `squeue`를 실행하면 아래와 같은 출력을 볼 수 있다.

```
$ squeue
         JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
       1648472    class1     loop  shpc028 CG       0:37      1 a05
```

(c)
```
$ srun -p class1 -N 2 hostname
a05
a06
```

srun은 작업을 실행하는 명령어로, -p 옵션으로 파티션을 지정하고 -N 옵션으로 할당할 노드의 수를 정할 수 있다. hostname은 현재 시스템의 호스트 이름을 확인하는 명령이므로, 출력 결과인 a05와 a06은 각각 해당 노드에서 hostname 명령이 실행됨을 의미한다.

(d)
```
shpc028@login2:~$ lscpu
Architecture:                    x86_64
CPU op-mode(s):                  32-bit, 64-bit
Byte Order:                      Little Endian
Address sizes:                   46 bits physical, 48 bits virtual
CPU(s):                          20
On-line CPU(s) list:             0-19
Thread(s) per core:              1
Core(s) per socket:              10
Socket(s):                       2
NUMA node(s):                    2
Vendor ID:                       GenuineIntel
CPU family:                      6
Model:                           85
Model name:                      Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz
Stepping:                        7
CPU MHz:                         1000.369
CPU max MHz:                     3200.0000
CPU min MHz:                     1000.0000
BogoMIPS:                        4800.00
Virtualization:                  VT-x
L1d cache:                       640 KiB
L1i cache:                       640 KiB
L2 cache:                        20 MiB
L3 cache:                        27.5 MiB
NUMA node0 CPU(s):               0-9
NUMA node1 CPU(s):               10-19
Vulnerability Itlb multihit:     KVM: Vulnerable
Vulnerability L1tf:              Not affected
Vulnerability Mds:               Not affected
Vulnerability Meltdown:          Not affected
Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Vulnerability Spectre v1:        Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2:        Mitigation; Enhanced IBRS, IBPB conditional, RSB filling
Vulnerability Srbds:             Not affected
Vulnerability Tsx async abort:   Mitigation; TSX disabled
Flags:                           fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 c
                                  sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_per
                                 xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cp
                                  fma cx16 xtpr pdcm pcid dca sse4_1 sse4_2 x2apic movbe popcnt tsc_deadli
                                 rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb cat_l3 cdp_l3 invpcid_si
                                 p ibrs_enhanced tpr_shadow vnmi flexpriority ept vpid ept_ad fsgsbase tsc
```

```
                                erms invpcid cqm mpx rdt_a avx512f avx512dq rdseed adx smap clflushopt cl
                                w avx512vl xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_t
                                a arat pln pts hwp hwp_act_window hwp_epp hwp_pkg_req pku ospke avx512_vn
                                apabilities
shpc028@login2:~$ srun -p class1 -N 1 lscpu
Architecture:                   x86_64
CPU op-mode(s):                 32-bit, 64-bit
Byte Order:                     Little Endian
Address sizes:                  46 bits physical, 48 bits virtual
CPU(s):                         64
On-line CPU(s) list:            0-63
Thread(s) per core:             2
Core(s) per socket:             16
Socket(s):                      2
NUMA node(s):                   2
Vendor ID:                      GenuineIntel
CPU family:                     6
Model:                          85
Model name:                     Intel(R) Xeon(R) Silver 4216 CPU @ 2.10GHz
Stepping:                       7
CPU MHz:                        800.004
CPU max MHz:                    3200.0000
CPU min MHz:                    800.0000
BogoMIPS:                       4200.00
Virtualization:                 VT-x
L1d cache:                      1 MiB
L1i cache:                      1 MiB
L2 cache:                       32 MiB
L3 cache:                       44 MiB
NUMA node0 CPU(s):              0-15,32-47
NUMA node1 CPU(s):              16-31,48-63
Vulnerability Itlb multihit:    KVM: Mitigation: Split huge pages
Vulnerability L1tf:             Not affected
Vulnerability Mds:              Not affected
Vulnerability Meltdown:         Not affected
Vulnerability Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl and seccomp
Vulnerability Spectre v1:       Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Vulnerability Spectre v2:       Mitigation; Enhanced IBRS, IBPB conditional, RSB filling
Vulnerability Srbds:            Not affected
Vulnerability Tsx async abort:  Mitigation; TSX disabled
Flags:                          fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 c
```

`lscpu` 명령은 현재 시스템의 cpu 정보를 보여주는 명령으로, 이를 로그인 노드인 `login2`에서 직접 실행하면 로그인 노드의 cpu 정보(아키텍쳐, 소켓/코어/스레드 수, 모델명, 캐시 정보, 명령어 세트 등)을 확인할 수 있다.

`srun -p class1 -N 1 lscpu`는 `lscpu`를 계산 노드에서 실행하는 명령으로, 계산 노드의 cpu 정보를 확인할 수 있다.

두 명령의 출력이 다른 이유는 각 노드의 물리적인 하드웨어가 다르기 때문이다. 두 명령의 결과에서 로그인 노드와 계산 노드의 코어 수, CPU 모델명, 캐시 용량 등이 다른 것을 확인할 수 있다.